Big Data and Machine Learning for Applied Economics

Week 05: Trees, Forests, and Boosting: Split, Average, and Correct

Eduard F. Martinez Gonzalez

Department of Economics, Universidad Icesi

October 13, 2026

Previously...

- **Week 3, the judge and the knob:** cross-validation chooses without touching the test set; the penalty shrinks and switches off, and pays when columns compete with rows. Country-wide, the simple model won —census + department dummies, test error 15.6— leaving one debt: slopes change across departments and we never wrote them.
- **Week 4, probability and decision:** the logit estimates $P(\text{win} \mid x)$; costs set the cutoff ($\tau^* = 0.83$ with the campaign's 5-to-1 prices: cost drops from 1,141 to 695) and the AUC ranks without fixing one (census + dummies logit: 0.916).

What we bring

Two marks to beat with the same data: in regression, a test error of 15.6; in classification, an AUC of 0.916. And one debt: the relationship changes from one department to the next, and nobody wrote those slopes.

Today's question: can the model find on its own where the relationship changes?

Same polling stations, same two questions; what changes is who writes the flexibility

- **What we never wrote by hand:** in week 3 the relationship changed across departments and we paid the cheap price: a dummy moves the level, not the slope. What if education matters differently in rural areas? With 37 variables there are 666 pairwise interactions and 7,770 three-way ones: nobody writes them.
- **Today's idea:** split the stations into similar groups and predict with each group's average. A split inside another split is an interaction nobody wrote.
- **One family, three methods:** the **tree** splits, the **forest** averages hundreds of trees, and **boosting** adds small trees that correct the previous ones. They work for both questions.

The takeaway

Split, average, and correct. A tree finds interactions on its own but is unstable; averaging it (forests) or correcting it little by little (boosting) turns it into the best model of the course.

1 The tree: split and average

2 The forest: average to reduce variance

3 Boosting: learning slowly

4 The horse race: did the machine find the interactions?

5 What if it is a classification question?

6 Closing

Back to Cali: what if, instead of a line, we split?

Ten training stations from the metro area, drawn at random, sorted by higher education

Polling station	Municipality	Higher ed. (%)	Vote (%)	RSS if split below
La Dolores	Palmira	6.3	67.0	638
Ayacucho La Buitrera	Palmira	7.5	67.4	624
Escuela Bartolomé Loboguerrero	Cali	7.8	72.8	547
La Castilla	Cali	9.4	74.1	401
Liceo Técnico Comercial Litecom	Jamundí	10.5	64.2	411
Esc. Nuestra Señora de los Remedios	Cali	11.6	58.7	501
IE Mater Dei	Palmira	14.5	63.4	495
IE José Celestino Mutis	Cali	16.7	72.0	**237**
Pance La Vorágine	Cali	29.8	55.9	316
Inst. Univ. Antonio José Camacho	Cali	42.4	47.0	

With no split, the best constant is the mean and what remains unexplained is its RSS: 646. **One split** divides the stations in two, and each side uses its own mean. With ten stations there are nine possible splits.

Question for the room: where would you split?

Back to Cali: what if, instead of a line, we split?

Ten training stations from the metro area, drawn at random, sorted by higher education

Polling station	Municipality	Higher ed. (%)	Vote (%)	RSS if split below
La Dolores	Palmira	6.3	67.0	638
Ayacucho La Buitrera	Palmira	7.5	67.4	624
Escuela Bartolomé Loboguerrero	Cali	7.8	72.8	547
La Castilla	Cali	9.4	74.1	401
Liceo Técnico Comercial Litecom	Jamundí	10.5	64.2	411
Esc. Nuestra Señora de los Remedios	Cali	11.6	58.7	501
IE Mater Dei	Palmira	14.5	63.4	495
IE José Celestino Mutis	Cali	16.7	72.0	**237**
Pance La Vorágine	Cali	29.8	55.9	316
Inst. Univ. Antonio José Camacho	Cali	42.4	47.0	

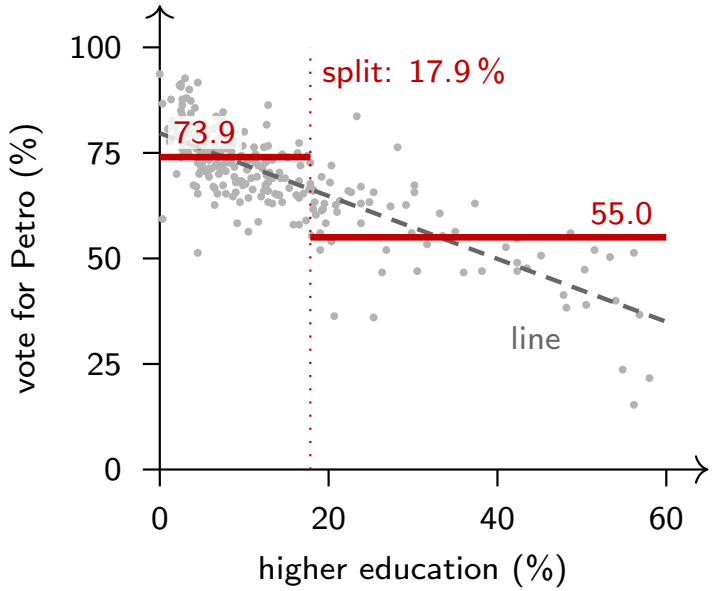
With no split, the best constant is the mean and what remains unexplained is its RSS: 646. **One split** divides the stations in two, and each side uses its own mean. With ten stations there are nine possible splits.

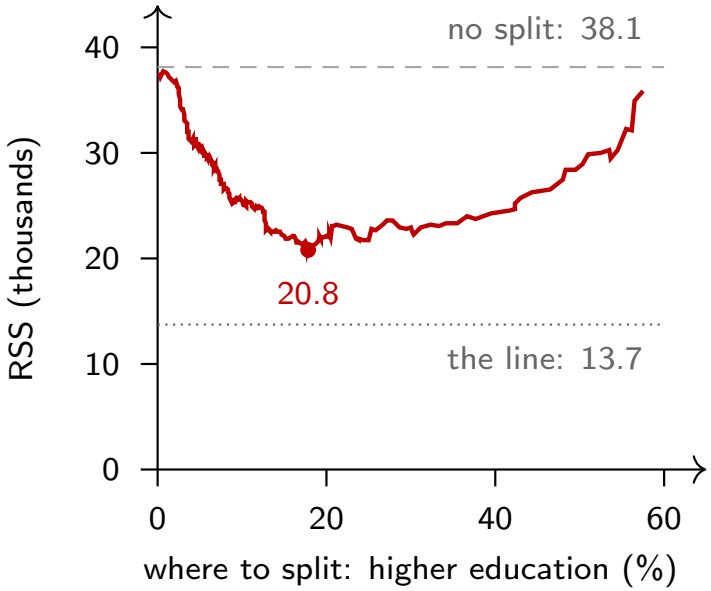
Question for the room: where would you split?

At 23.3 %: the 8 least-educated stations (mean 67.4) and the 2 most educated (51.5); the RSS falls from 646 to 237. The RSS has two valleys: the other one is at 9.9 % (401).

The same split with all 257 stations

The machine runs the same computation over every possible split and keeps the one that lowers the RSS the most

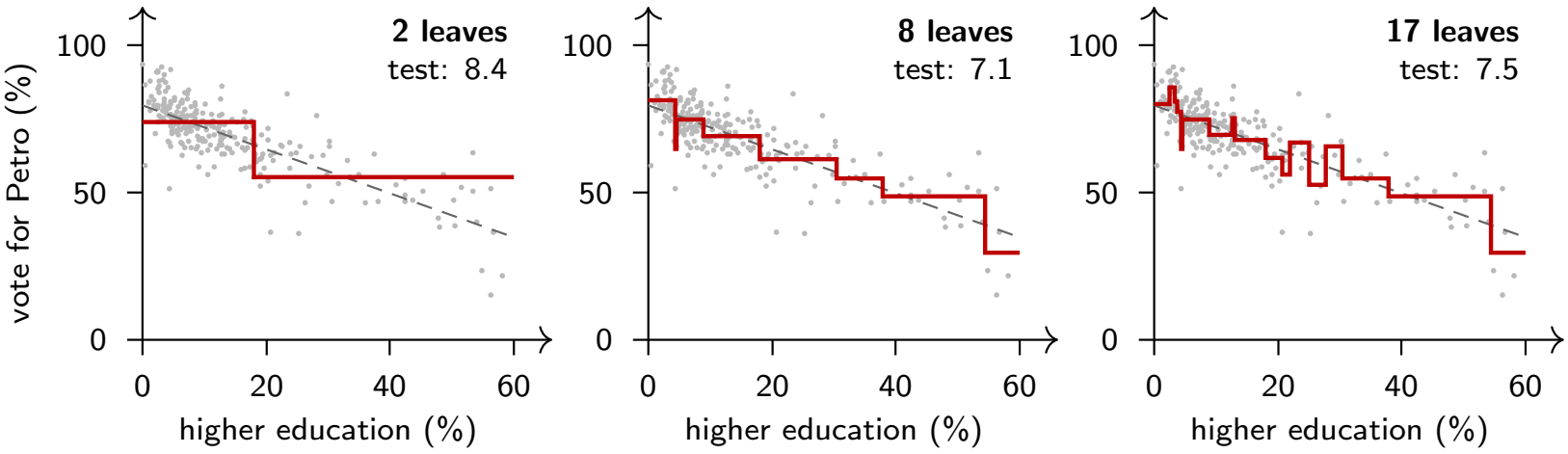




- The best split lands at 17.9 % higher education: 192 stations on the left, with a mean vote of 73.9, and 65 on the right, with 55.0. The RSS falls 45 %, from 38.1 to 20.8 thousand.
- Week 2's line leaves 13.7 thousand: with a single variable and a nearly linear relationship, one split is not enough. What if we split again?

Splitting again: a staircase

Each half is split again with the same rule. In red, the tree; dashed, the line



- Each step is a **leaf**: a group of stations predicted with its own mean.
- **On the training set** the error always falls with more leaves: 9.0 with 2 leaves, 6.6 with 8, and 6.2 with 17. **On the test set** it falls from 8.4 to 7.1 and then rises again to 7.5.
- None of them beats week 2's line, which leaves 6.8 on the test set: in Cali the relationship is nearly a straight line. How many leaves should we keep? At the end of this part.

The model: one constant per region

Split the stations into groups and predict with each group's mean (ISL, section 8.1.1)

$$\hat{f}(x) = \sum_{h=1}^{H} c_h \, \mathbb{I}\{x \in R_h\} \qquad \text{with } R_1, \ldots, R_H \text{ chosen to minimize the RSS}$$

- Each region R_h is a group of stations (a leaf), and its prediction is the best constant: the mean of its stations, $\hat{c}_h = \bar{y}_{R_h}$.
- **The problem:** there are too many ways to form the groups; they cannot all be tried. **The solution:** one split at a time. A variable j and a cutpoint s leave two groups, $R_1 = \{x : x_j < s\}$ and $R_2 = \{x : x_j \geq s\}$, and we look for the pair that minimizes

$$\min_{j,\, s} \sum_{i:\, x_i \in R_1} (y_i - \bar{y}_{R_1})^2 + \sum_{i:\, x_i \in R_2} (y_i - \bar{y}_{R_2})^2$$

- Once that split is made, the same thing is repeated inside each group, and so on.
- **It is a greedy method:** at each step it takes the best split available right now, without checking whether another one, worse today, would give a better tree later.

The algorithm, step by step

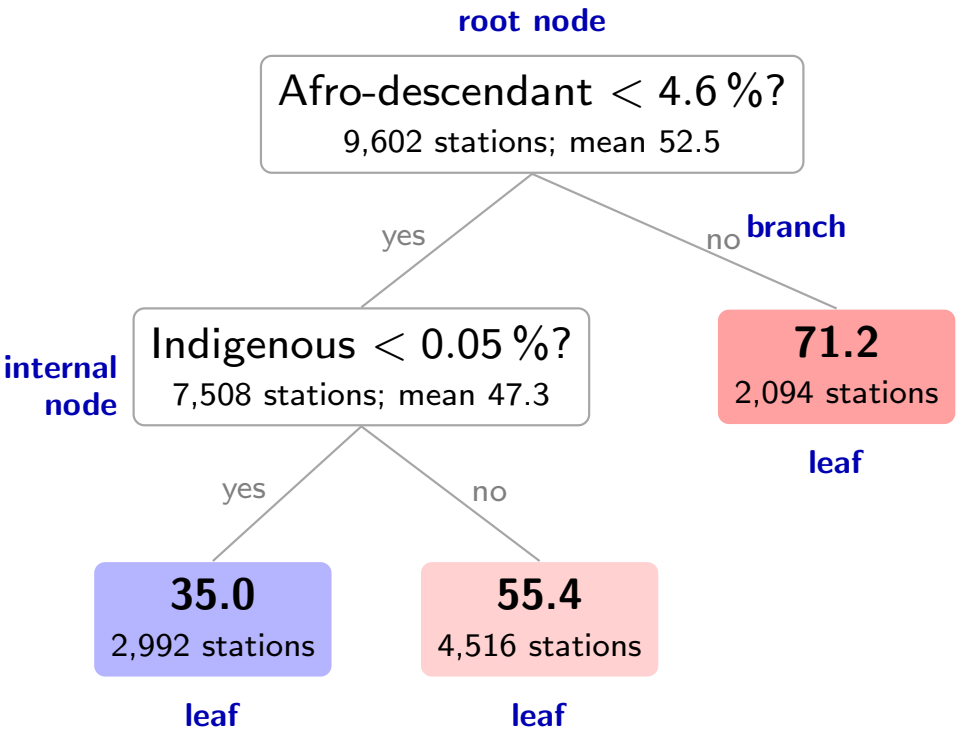
The candidate splits come from the data and all of them are tried: nothing is random

1. **It starts with every station in a single group.** The prediction is the mean (country-wide, 52.5) and its RSS is the starting point. There is no seed: with the same data, the tree always comes out the same.
2. **The candidate splits come from the data.** Sort the values of a variable; every midpoint between two consecutive values is a candidate. With Cali's ten stations there were nine. Nobody decides how far to move up or down: **all** of them are tried.
3. **Each candidate is scored by its RSS:** one mean on each side and the sum of squared errors. And this is done for **every variable**: country-wide, 37 variables and 208,236 candidates in total.
4. **The split with the lowest RSS wins.** Variable and cutpoint come out together: the tree does not pick "the variable first". Country-wide the winner was the Afro-descendant share, at 4.6 %.
5. **Repeat inside each group, separately.** Nobody chooses which group goes next: all of them keep splitting, until a group has fewer than 20 stations or no split improves the fit.

To review: the video Regresión con árboles de decisión (Codificando Bits, in Spanish).

The parts of a tree: root, nodes, and leaves

The country-wide tree with its first two splits: the vocabulary, before going on

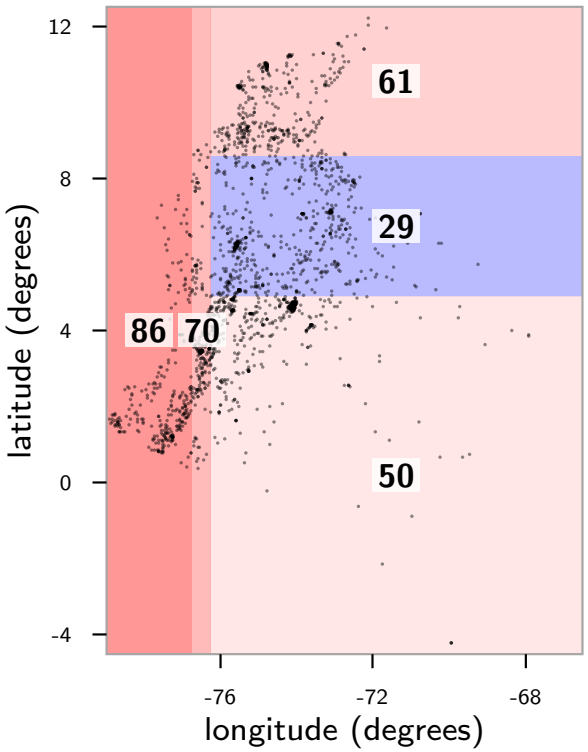


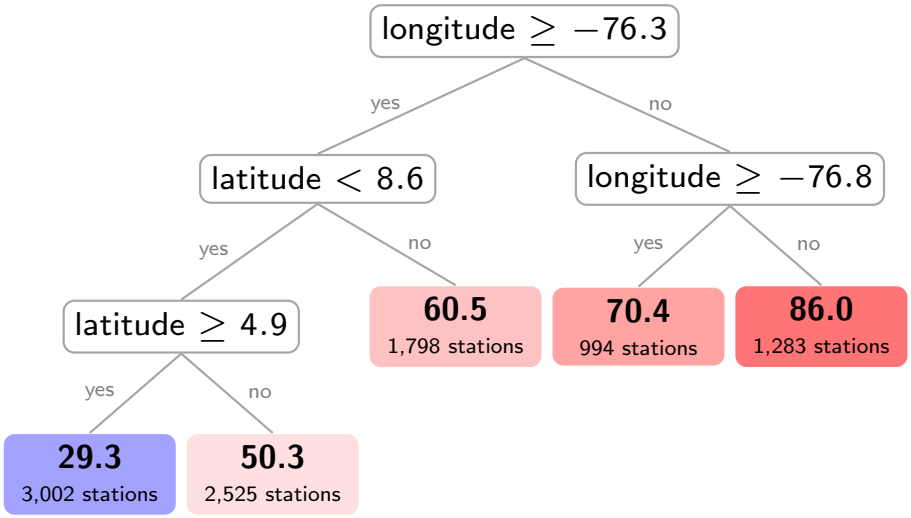
- **Node:** each box; it is a group of stations.
- **Root node:** the first one. It holds every station and asks the first question.
- **Internal nodes:** those that ask another question and split into two **branches** (yes and no).
- **Leaves** (terminal nodes): those that are not split any further. Their mean is the prediction.
- **Size:** the number of leaves (here, 3). **Depth:** the questions from the root to the farthest leaf (here, 2).

To predict a new station, answer the questions from the root until you land in a leaf: a station with a 1 % Afro-descendant share and a 3 % Indigenous share lands in the 55.4 leaf. That is its predicted vote.

Splitting the map: the tree and its rectangles

A tree on the country's longitude and latitude: the same model in two drawings (ISL, figure 8.3)



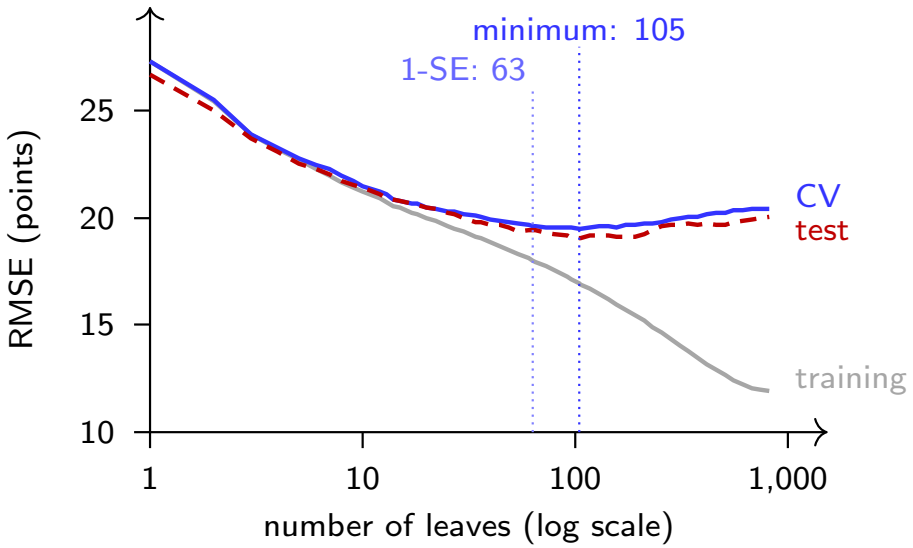


- **The same tree in two drawings:** each leaf of the diagram (right) is a rectangle on the map (left); the number is the mean vote of its stations.
- **Five leaves, five rectangles:** the Pacific–Nariño strip stands alone, at 86.0. Test error: 19.2; linear regression on the 37 census variables, 19.8.

How far should it grow? Trees overfit too

Cost-complexity pruning: a price per leaf, chosen by CV (ISL, algorithm 8.1)

$$\min_{T \subseteq T_0} \sum_{h=1}^{|T|} \sum_{i:\, x_i \in R_h} (y_i - \bar{y}_{R_h})^2 \;+\; \alpha\, |T|$$



- Grown until leaves hold 7 stations, the tree on the 37 census variables has 819 leaves: training error 11.9, test error 20.0 (golden rule #1).
- α is the price per leaf, like the Lasso's λ: $\alpha = 0$ gives the full tree; large α, the root. The weakest link goes first.
- Week 3's cross-validation (CV) picks 105 leaves: 19.1 on test; with the one-standard-error rule, 63 (19.4).

The tree's problem: it memorizes and it is unstable

The country-wide tree on the 37 census variables, grown larger and larger

	Error (RMSE)	
Leaves	training	test
2	25.5	25.0
8	21.6	21.7
63	18.0	19.4
105	16.9	**19.1**
819	11.9	20.0

- **Small, it falls short:** with 2 or 8 leaves the error is high in both columns.
- **Large, it memorizes:** with 819 leaves the training error drops to 11.9 and the test error rises to 20.0.
- **Pruned, it is not enough:** the best one (105 leaves) leaves 19.1; linear regression on the same variables, 19.8.

And it is unstable. We grew the tree twenty times, each time on a different sample of the same training set (the stations are drawn again: some repeat and others are left out). The first split does not change (the Afro-descendant share, 20 out of 20), but the ones below do: the prediction for the same station moves 9.2 points from one tree to the next. For week 3's best linear regression, 1.1.

How can we predict better without overfitting?

None of the twenty trees, on its own, gets below 20.1 on the test set. **The average of the twenty: 17.3.** There is no need to pick the best tree: average many. That is a forest.

1 The tree: split and average

2 The forest: average to reduce variance

3 Boosting: learning slowly

4 The horse race: did the machine find the interactions?

5 What if it is a classification question?

6 Closing

Bagging: averaging many trees

Breiman (1996); ISL, section 8.2.1

The recipe: (1) build a new sample by drawing n stations from the training set **with replacement** (some come up more than once, others are left out); (2) fit a **large**, unpruned tree to it; (3) repeat B times and average the B predictions:

$$\hat{f}_{\text{bag}}(x) \;=\; \frac{1}{B}\sum_{b=1}^{B} \hat{f}^{*b}(x)$$

- **Why large trees:** each one errs a lot, but not in a fixed direction (low bias, high variance): when averaged, their errors offset each other.
- **How far averaging helps:** if the B trees were independent, each with variance σ^2, the average would have σ^2/B. But they come from the same data and look alike: with correlation ρ between any two trees,

$$\text{Var}\Big(\frac{1}{B}\sum_{b=1}^{B} \hat{f}^{*b}(x)\Big) \;=\; \rho\,\sigma^2 \;+\; \frac{1-\rho}{B}\,\sigma^2 \qquad \text{(appendix A1)}$$

- With more trees the second term vanishes; the first does not: **the limit is set by how similar the trees are**.

ISL calls drawing with replacement the *bootstrap*; hence the name, *bootstrap aggregation*.

The random forest: making the trees less alike

At each split only m random variables compete (Breiman, 2001; ISL, section 8.2.2)

- **The problem with bagging:** every tree sees almost the same data and makes almost the same splits: the Afro-descendant share was the first split in 20 out of 20.
- **The solution:** at each split, m of the p variables are drawn at random and only those compete. Sometimes the strongest one is left out: each tree ends up using a **different combination of variables**.
- **Two letters, two things:** B is how many trees are averaged; m, how many variables compete at each split. With $m = 37$ (all of them) it is bagging; the usual choice is $m \approx p/3$ (here, 12).
- **The result** with $m = 12$: each tree is a little worse (23.8 versus 23.4), but they look less alike (correlation of 0.54 versus 0.58) and the average improves (from 15.8 to 15.6).

Question for the room

How can the average improve if every single tree gets worse?

The random forest: making the trees less alike

At each split only m random variables compete (Breiman, 2001; ISL, section 8.2.2)

- **The problem with bagging:** every tree sees almost the same data and makes almost the same splits: the Afro-descendant share was the first split in 20 out of 20.
- **The solution:** at each split, m of the p variables are drawn at random and only those compete. Sometimes the strongest one is left out: each tree ends up using a **different combination of variables**.
- **Two letters, two things:** B is how many trees are averaged; m, how many variables compete at each split. With $m = 37$ (all of them) it is bagging; the usual choice is $m \approx p/3$ (here, 12).
- **The result** with $m = 12$: each tree is a little worse (23.8 versus 23.4), but they look less alike (correlation of 0.54 versus 0.58) and the average improves (from 15.8 to 15.6).

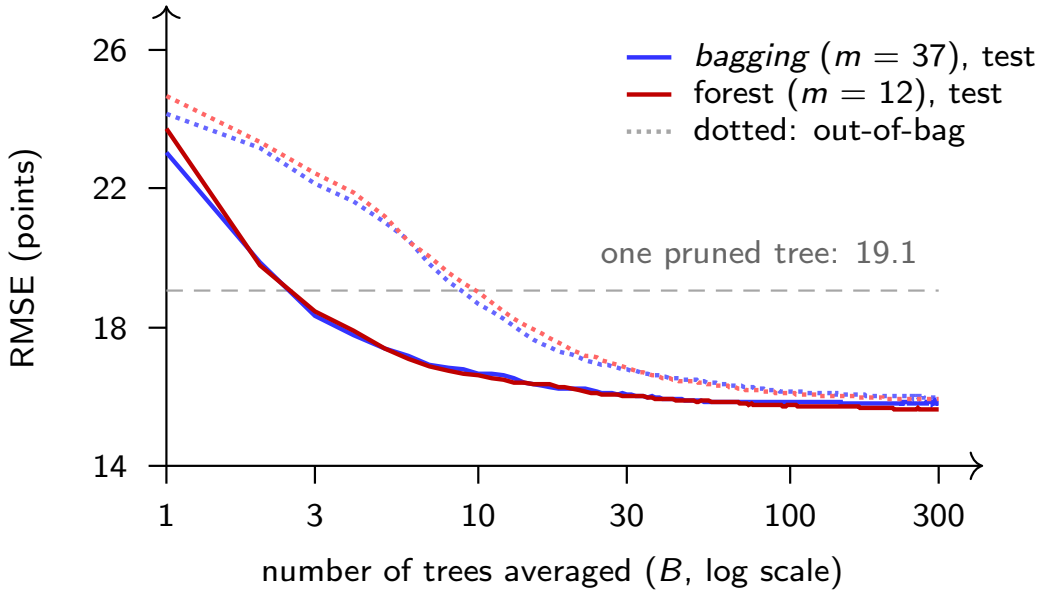
Question for the room

How can the average improve if every single tree gets worse?

Because the average gains more from trees that err differently. It is Condorcet's jury theorem (1785): if each juror is right 60 % of the time, a majority of three is right 65 % of the time and a majority of eleven, 75 %, as long as they do not all make the same mistake. To see it move: mlu-explain.github.io/random-forest.

Bad trees, good average

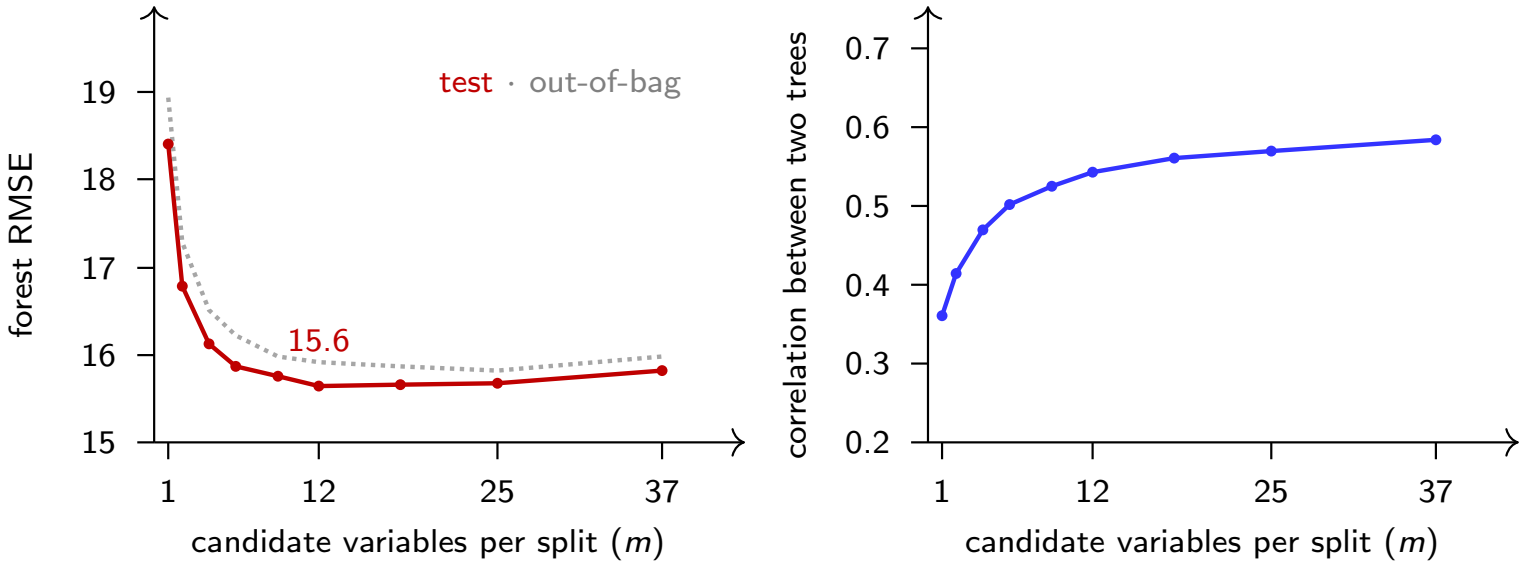
Error by number of trees averaged, with the 37 census variables (ISL, figure 8.8)



- **How to read it:** blue, bagging (all 37 variables compete at each split); red, the random forest (only 12 drawn at random). Solid lines: the test error; dotted: out-of-bag, on the stations each tree did not use (appendix A7).
- A single large tree: 23.1 on the test set, worse than the pruned one (19.1). The average of 10: 16.7. Of 300: 15.8. Linear regression on the same variables: 19.8.
- **More trees never hurt the average:** B needs no tuning; add enough for the curve to flatten out.

How many variables per split? Choosing m

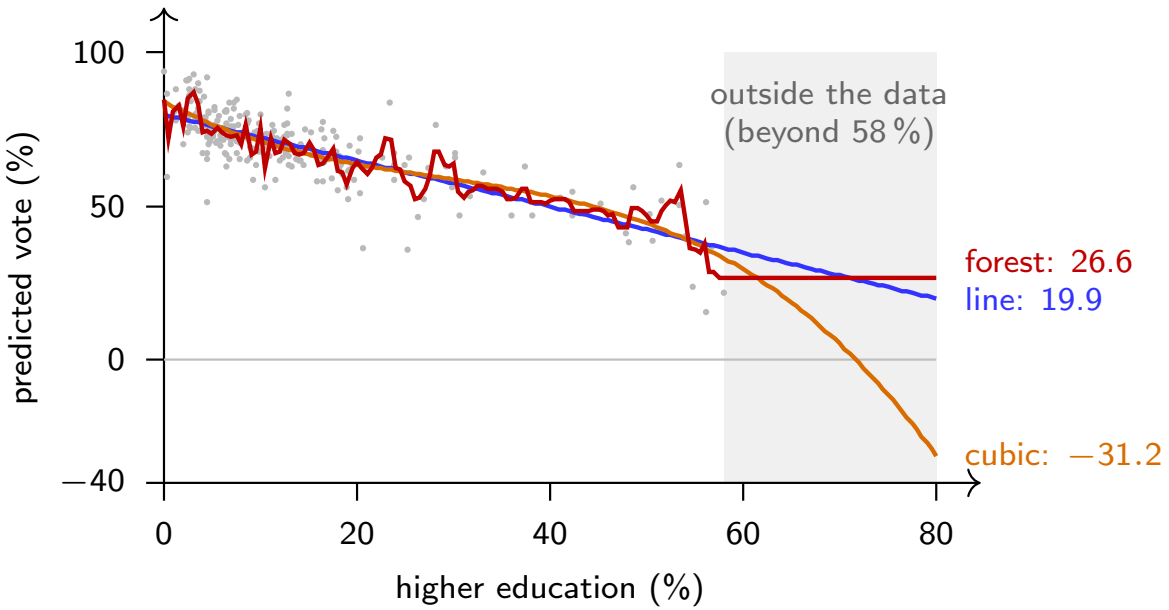
The forest changes little with m: that is why it works almost without tuning



- **How to read it:** on both horizontal axes, m. Left: the forest's error, in red on the test set and in gray on the stations each tree did not use (out-of-bag). Right: how similar two trees of the same forest are (their correlation).
- $m = 1$**,** one random variable at each split: the trees barely resemble each other (correlation of 0.36), but each one is very bad (25.6); the forest ends up at 18.4.
- $m = 37$ is bagging: the best trees and the most alike. **In between** the curve is almost flat (15.6 with $m = 12$; the best out-of-bag is $m = 25$): with $m \approx p/3$ tuning is almost never needed.

What the forest will not do: leave the data

Week 2's extrapolation, again: three models on the metro area's higher education



- **How to read it:** the dots are the metro area's stations; in the gray band (beyond 58 % higher education) there are none. On the right, what each model predicts at 80 %.
- **The forest averages votes it has already seen:** outside the data it stays flat (26.6). The line follows its slope (19.9) and the cubic drops to −31.2, an impossible vote. The forest, at least, does not make things up.

1 The tree: split and average

2 The forest: average to reduce variance

3 Boosting: learning slowly

4 The horse race: did the machine find the interactions?

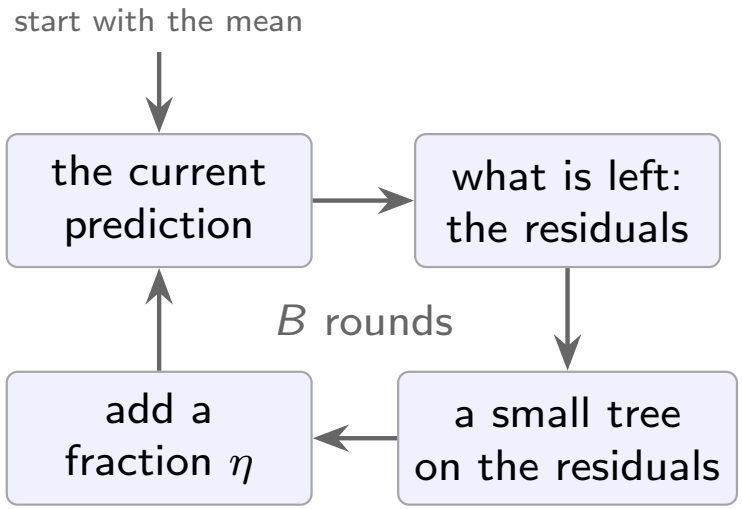
5 What if it is a classification question?

6 Closing

Boosting: small trees that correct what is left

The algorithm for regression (ISL, algorithm 8.2)

1. **Start with the mean:** every station gets the same prediction, $\hat{f}(x) = \bar{y}$.
2. **Measure what is left:** each station's residual, $r_i = y_i - \hat{f}(x_i)$.
3. **Fit a small tree to the residuals,** not to the vote: it finds the groups of stations where the model is most wrong.
4. **Add only a fraction η of that tree** (say, 10 %; ISL calls it λ): $\hat{f}(x) \leftarrow \hat{f}(x) + \eta\, \hat{f}^b(x)$.
5. **Go back to step 2** with the new prediction; and so on for B rounds.

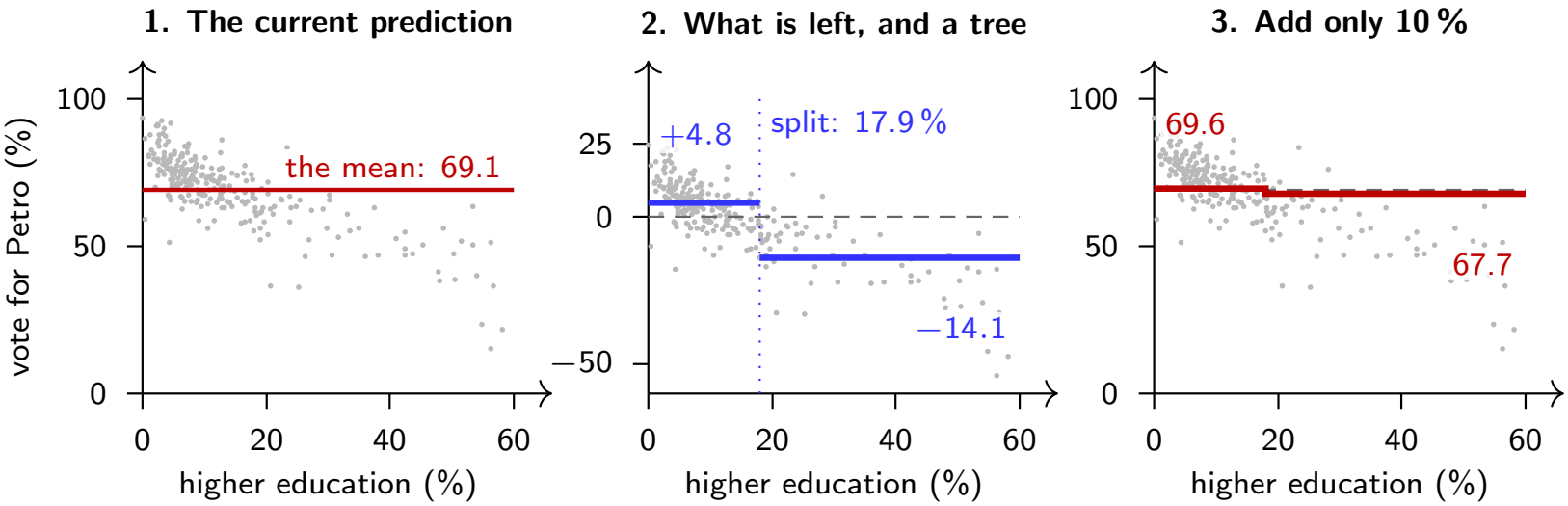


$$\hat{f}(x) \;=\; \bar{y} \;+\; \eta\, \hat{f}^1(x) \;+\; \eta\, \hat{f}^2(x) \;+\; \ldots \;+\; \eta\, \hat{f}^B(x)$$

The forest averages large trees, each grown on its own; **boosting** adds up small trees, in sequence: each one corrects what the previous ones left.

One round, by hand, in Cali

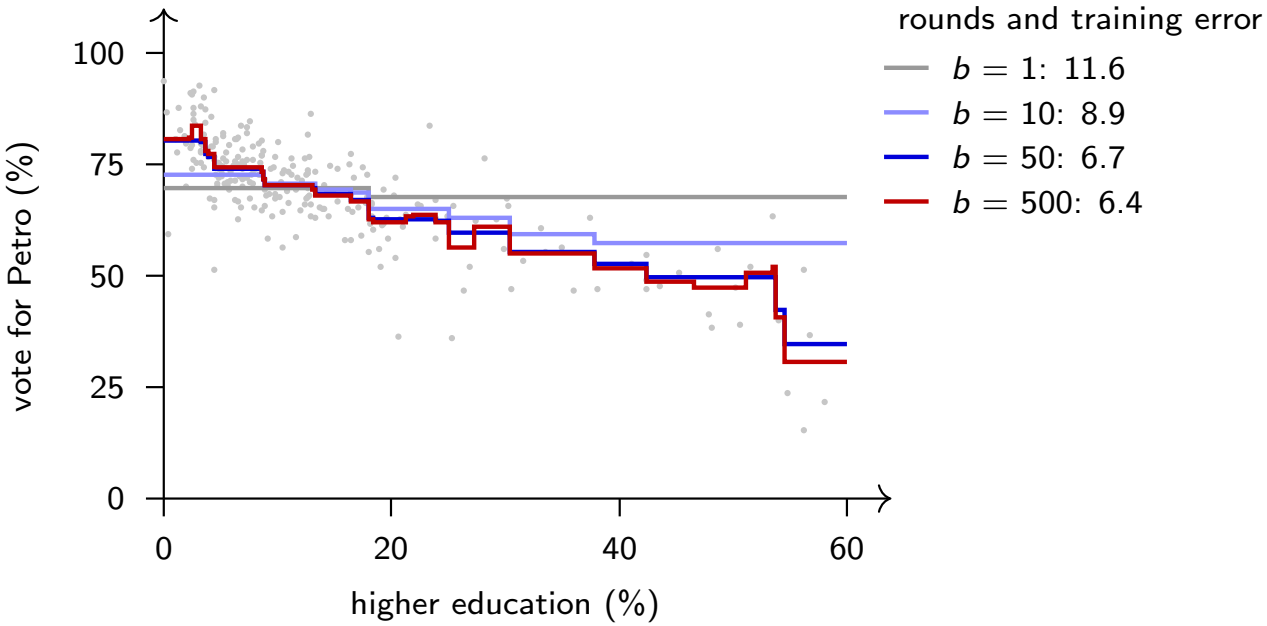
The metro area's 257 stations, single-split trees, and $\eta = 0.1$



- **1.** Start with the mean, 69.1 for everyone (red line). Training error: 12.2.
- **2.** What is left at each station: its vote minus 69.1. A single-split tree (blue) splits those residuals at 17.9 %: on the left the model is 4.8 points short on average; on the right it overshoots by 14.1.
- **3.** Add 10 %: $69.1 + 0.5 = 69.6$ on the left and $69.1 - 1.4 = 67.7$ on the right. The error falls to 11.6.
- **Rounds 2 and 3** repeat the computation with the new residuals (round 2 splits at 25.1 %): the error falls to 11.2 and then 10.8. Each round moves little: that is why many are needed.

Learning slowly, in Cali

The same computation, up to 500 rounds: single-split trees on higher education, with $\eta = 0.1$



- **How to read it:** each line is the model after b rounds; on the right, its training error.
- With one round, almost the mean; with 10, a timid step; with 50, the shape; with 500, a fine staircase chasing the points.
- The training error only falls (from 11.6 to 6.4). The test error bottoms out at 7.1 (round 48) and by round 500 it is at 7.3: **unlike the forest, here more rounds do overfit**.

Why fit the residuals?

The residual tells the model which way to correct, and by how much, at each station

- **The residual is an instruction.** If the model predicts 69.1 and the station voted 80, the residual is $+10.9$: go up there. If it voted 50, it is -19.1: go down there.
- **The small tree turns it into a rule:** it does not correct station by station (that would be memorizing); it looks for groups of stations with similar residuals and corrects the whole group.
- **The fraction η is the step size:** correct a little, measure again what is left, and correct once more.
- **Hence the name, gradient boosting.** With squared error, $L_i = \frac{1}{2}\big(y_i - f(x_i)\big)^2$, the residual is its derivative with the sign flipped:

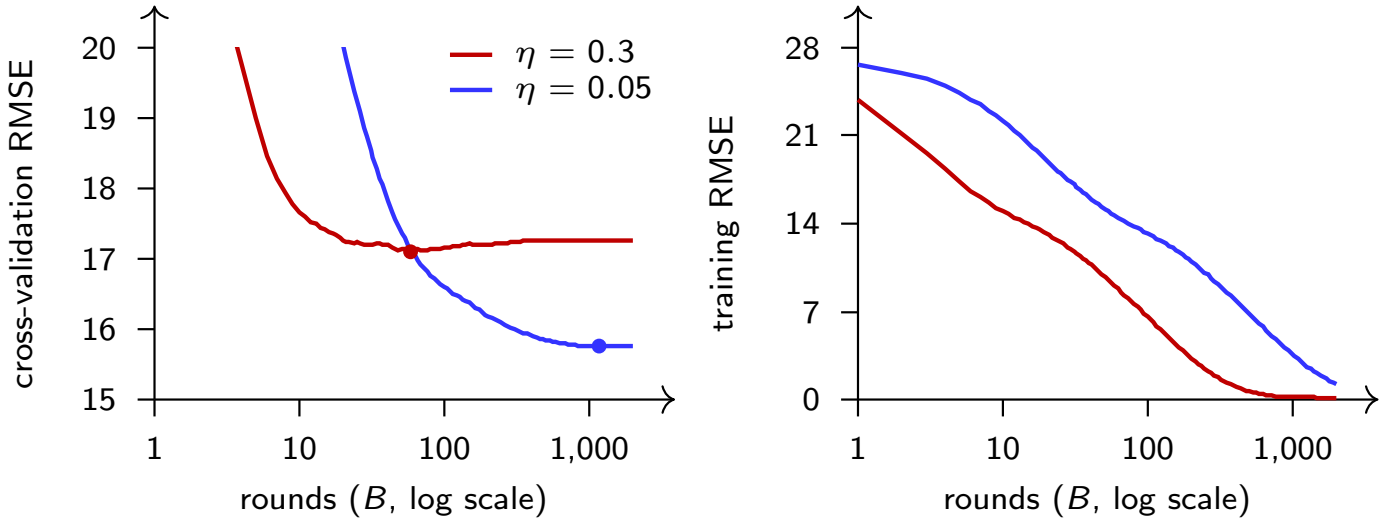
$$-\frac{\partial L_i}{\partial f(x_i)} \;=\; y_i - f(x_i) \;=\; r_i.$$

  Moving in the direction of the residual lowers the error along the steepest path: gradient descent, with steps of size η.

With another error measure only the "residual" changes: for `gana_petro`, $y_i - \hat{p}_i$ (appendix A2).

How many rounds and how fast? Choosing B and η

Country-wide, the 37 census variables: cross-validation error (left) and training error (right)



- **How to read it:** on both horizontal axes, the rounds B. Red: 30 % of each tree is added ($\eta = 0.3$). Blue: 5 % ($\eta = 0.05$).
- **B, the rounds:** on the training set (right) the error always falls. Under cross-validation (left) it falls, hits a minimum, and rises again: from there on, new rounds only memorize. Stop at the minimum (*early stopping*).
- **η, the rate:** with 0.3 the minimum arrives at round 58 (17.1); with 0.05 it arrives at round 1,167 and lower (15.8). Slower is better; the price is computing time.

Tree, forest, and boosting, side by side

What each one does, what has to be chosen, and how

	A tree	**Random forest**	**Boosting**
How many trees	one	hundreds, each on its own	hundreds or thousands, in sequence
How large	medium: it gets pruned	large, unpruned	small
How they combine	—	averaged	added up: each corrects the previous ones
What to choose	the number of leaves	almost nothing: $m \approx p/3$	the rounds B, the rate η, and the size d
How to choose it	cross-validation	the out-of-bag error	cross-validation, with early stopping
More trees	—	never overfit	do overfit
Can you read it?	yes	no	no
In R	`rpart`	`randomForest`	`xgboost`

Today: forests with $m = 12$ and leaves of 5 stations; boosting with $\eta = 0.05$ and $d = 6$. All are compared on the same test set, opened only once (appendix A6).

1 The tree: split and average

2 The forest: average to reduce variance

3 Boosting: learning slowly

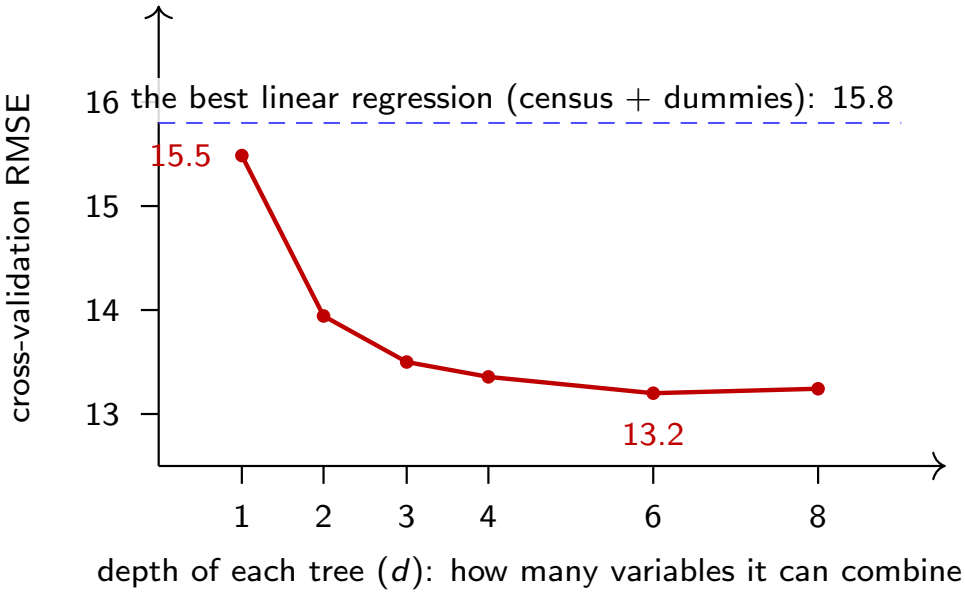
4 The horse race: did the machine find the interactions?

5 What if it is a classification question?

6 Closing

How much are interactions worth?

The same boosting on census + department; only d, the size of each tree, changes



- **What d is:** how many splits each small tree chains together, that is, how many variables it can combine. The dashed line is week 3's best linear regression.
- $d = 1$**:** one split per tree, a single variable: the model cannot combine variables. It leaves 15.5, almost the same as linear regression (15.8).
- $d = 2$ already combines pairs: 13.9. $d = 6$**:** 13.2. Between $d = 1$ and $d = 6$ there are 2.3 points: that is what interactions are worth, and the model found them on its own. **The answer to today's question is yes.**

The horse race: every model, the same test set

Cross-validation (CV) and test RMSE (2,398 stations, opened only once)

Variables	Model	CV	Test
—	the mean (the benchmark)	27.3	26.7
Census (37)	linear regression	20.3	19.8
	a pruned tree (105 leaves)	19.5	19.1
	forest	15.9	15.6
	boosting	15.8	15.7
Census + department	linear regression (week 3's best)	15.8	15.6
	a pruned tree (98 leaves)	15.7	15.7
	forest	13.3	12.9
	boosting	13.2	**12.8**
+ coordinates	forest	12.3	12.1
	boosting	12.2	**12.1**

For the forests, the CV column is the out-of-bag error (appendix A7).

- **A single tree does not beat linear regression** (15.7 versus 15.6). Hundreds of trees do.
- **With the same variables,** forest and boosting beat the best linear regression by almost 3 points (12.9 and 12.8 versus 15.6). **Without knowing the department** they reach 15.6 and 15.7: the same as the linear regression that does know it.
- **With the coordinates** (longitude and latitude, as two more variables) they reach 12.1: where the station sits says something the census and the department do not.

1 The tree: split and average

2 The forest: average to reduce variance

3 Boosting: learning slowly

4 The horse race: did the machine find the interactions?

5 What if it is a classification question?

6 Closing

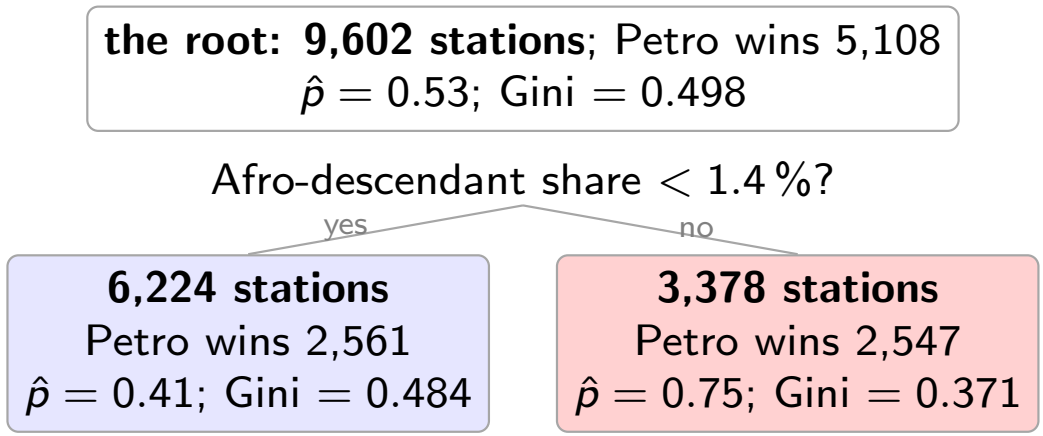
Classifying with a tree: same algorithm, another measure

Now y is `gana_petro`: 1 if Petro got more votes than Hernández (ISL, section 8.1.2)

Only two things change: (1) **the leaf predicts** the fraction of its stations that Petro won, $\hat{p}$, a probability; (2) **a split is scored** by how **mixed** the two groups end up, not by the RSS.

The Gini index of a group, $G = 2\,\hat{p}\,(1 - \hat{p})$, measures the mix: it is 0 if the same candidate won all its stations (a pure group) and 0.5 if they are split half and half.

Everything else is the same: all candidate splits are tried, and the one with the lowest average Gini across its two groups wins.



The country's first split: the average Gini of the two groups, weighted by their size, is $\frac{6{,}224}{9{,}602} \times 0.484 + \frac{3{,}378}{9{,}602} \times 0.371 = 0.444$, below the root's 0.498. No other split lowers it more.

Video: Clasificación con árboles de decisión (Codificando Bits, in Spanish, 15 minutes). To move the split with your mouse: mlu-explain.github.io/decision-tree.

Does Petro win? The same models, the other question

Cost uses the campaign's prices from week 4 (cutoff 0.83)

Model	CV AUC	Test AUC	Brier	Cost
Logit, census + dummies (week 4)	0.920	0.916	0.115	695
a pruned tree, census + department	–	0.896	0.122	819
forest, census + department	0.942	0.943	0.096	617
boosting, census + department	0.944	0.941	0.096	609
forest, + coordinates	0.949	0.950	0.090	589
boosting, + coordinates	0.953	0.950	0.087	625

The *Brier score*, $\frac{1}{n}\sum_i(\hat{p}_i - y_i)^2$, is the MSE of the probabilities (appendix A7 of week 4).

- **How they give a probability:** in the forest each tree votes and $\hat{p}$ is the fraction of trees saying "Petro wins"; boosting adds up small trees on $y - \hat{p}$ (appendix A2).
- **A single tree ranks worse than the logit** (0.896 versus 0.916). Forest and boosting rank better (0.943 and 0.941), and the campaign spends less (617 and 609 versus 695).
- **Ranking is not deciding** (week 4): with coordinates, boosting has a better Brier than the forest (0.087 versus 0.090), but at the 0.83 cutoff it costs more (625 versus 589).

1 The tree: split and average

2 The forest: average to reduce variance

3 Boosting: learning slowly

4 The horse race: did the machine find the interactions?

5 What if it is a classification question?

6 Closing

When to use what

- **A linear regression:** if the relationship is nearly linear, there are few observations per variable, or you need to read coefficients or step carefully outside the data's range. In Cali the line won.
- **A tree:** to explain a simple rule (a risk traffic light), more than to predict.
- **A forest:** the best off-the-shelf model: almost nothing to tune and not very sensitive to what you choose. The first candidate with tabular data.
- **Boosting:** almost always the most accurate with tabular data, if η, d, and B are chosen by cross-validation. Across 45 medium-sized tabular datasets, tree ensembles still beat neural networks (Grinsztajn, Oyallon, and Varoquaux, 2022).

Getting ready for next class...

To get the most out of next session

This week's readings:

- Molnar (2025), *Interpretable Machine Learning*: the chapters on permutation importance, partial dependence, and SHAP. Open online book. Guide: each chapter lists advantages, disadvantages, and code; read the disadvantages twice.

Problem Set 1: due Tuesday, October 20 at 8:00 a.m., at the start of session 6.

Next session — opening the black box: today's winner is a model nobody can read: hundreds of trees and thousands of splits. Which variables does it use, how does the prediction move when one changes, and why does it predict what it predicts at a given station? Permutation importance, partial dependence, and SHAP, with their caveats.

Application in R: the same stations and the same two questions: a tree you can read, a forest, and a boosting model, against weeks 3 and 4's best, on the same test set.

Essential references

- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning* (2. ed.). Springer. Chap. 8. [ISL]
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2. ed.). Springer. Secs. 9.2, 10, and 15. [ESL]
- Breiman, L., Friedman, J., Olshen, R., & Stone, C. (1984). *Classification and Regression Trees*. Wadsworth.
- Breiman, L. (1996). Bagging Predictors. *Machine Learning*, 24(2), 123–140.
- Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
- Friedman, J. (2001). Greedy Function Approximation: A Gradient Boosting Machine. *Annals of Statistics*, 29(5), 1189–1232.
- Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of KDD '16*, 785–794.
- Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data? *NeurIPS Datasets and Benchmarks*.
- To review: Codificando Bits, Regresión con árboles de decisión and Clasificación con árboles de decisión (videos, in Spanish); MLU-Explain, Decision Trees and Random Forest (interactive explainers).
- Molnar, C. (2025). *Interpretable Machine Learning* (3. ed.). https://christophm.github.io/interpretable-ml-book/ [next session]
- Gelvez, J. D., Cardiles, A., Martínez-González, E. F., & Muñoz, M. (2026). How Predictable Is an Election? Working paper. [The source of the case data.]

7 Appendix: going deeper

A1. The average of correlated trees

Let $T_1, \ldots, T_B$ be the predictions of B trees at a point x, identically distributed, with $\mathrm{Var}(T_b) = \sigma^2$ and correlation ρ between any two:

$$\mathrm{Var}\Big(\frac{1}{B}\sum_b T_b\Big) \;=\; \frac{1}{B^2}\Big[\sum_b \mathrm{Var}(T_b) + \sum_{b\neq b'} \mathrm{Cov}(T_b, T_{b'})\Big]$$

$$= \;\frac{B\sigma^2 + B(B-1)\,\rho\,\sigma^2}{B^2} \;=\; \rho\,\sigma^2 + \frac{1-\rho}{B}\,\sigma^2$$

- As $B \to \infty$ what remains is $\rho\,\sigma^2$: more trees cannot push the variance below that floor.
- The forest lowers ρ with $m < p$, at the cost of slightly raising each tree's variance and bias. The right m balances the two (ESL, section 15.4).
- The average's bias is each tree's bias: that is why large, low-bias trees are averaged, not pruned ones.

A2. Gradient boosting with any loss

The general algorithm (Friedman, 2001; ESL, algorithm 10.3):

1. $f_0(x) = \arg\min_\gamma \sum_i L(y_i, \gamma)$: the mean under squared loss; the log-odds of the prevalence under the log-loss.
2. For $b = 1, \ldots, B$: (a) the pseudo-residuals $r_{ib} = -\left[\partial L(y_i, f(x_i))/\partial f(x_i)\right]_{f=f_{b-1}}$; (b) a tree with leaves R_{jb} fit to the r_{ib}; (c) in each leaf, the best step $\gamma_{jb} = \arg\min_\gamma \sum_{x_i \in R_{jb}} L(y_i, f_{b-1}(x_i) + \gamma)$; (d) $f_b(x) = f_{b-1}(x) + \eta \sum_j \gamma_{jb} \mathbb{I}\{x \in R_{jb}\}$.

Loss	$L(y, f)$	$-\partial L/\partial f$
Squared	$\frac{1}{2}(y - f)^2$	$y - f$
Absolute	$\lvert y - f \rvert$	$\text{sign}(y - f)$
Log-loss (f in log-odds)	$-yf + \log(1 + e^f)$	$y - \hat{p}$

- Under squared loss, (c) is the mean of the residuals in each leaf: ISL's algorithm 8.2.
- **Subsampling** (Friedman, 2002): each tree sees a fraction of the rows; as in the forest, it lowers the correlation between steps. Today, 80 %.

A3. xgboost: the second derivative and a price per leaf

In round b, with $g_i = \partial L/\partial \hat{y}_i$ and $h_i = \partial^2 L/\partial \hat{y}_i^2$ evaluated at the previous prediction (Chen and Guestrin, 2016):

$$\sum_i \left[g_i\, f_b(x_i) + \tfrac{1}{2}\, h_i\, f_b(x_i)^2 \right] \;+\; \gamma\, T \;+\; \tfrac{1}{2}\, \lambda \sum_{j=1}^{T} w_j^2$$

where T is the number of leaves and w_j the value of leaf j. With $G_j = \sum_{i \in j} g_i$ and $H_j = \sum_{i \in j} h_i$, the best leaf value and the gain from a split are

$$w_j^* = -\frac{G_j}{H_j + \lambda}, \qquad \frac{1}{2}\left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma$$

- **Squared loss:** $g_i = \hat{y}_i - y_i$ and $h_i = 1$, so $w_j^* = \sum_{i \in j} r_i/(n_j + \lambda)$: the leaf's mean residual, shrunken toward zero. Ridge in every leaf.
- **Log-loss:** $g_i = \hat{p}_i - y_i$ and $h_i = \hat{p}_i(1 - \hat{p}_i)$: the Hessian of week 4's logit. Each round is a Newton step.
- γ is a price per leaf, like pruning's α: a split enters only if it gains more than γ.

A4. Categorical variables and missing data in a tree

- A variable with q categories can be split in two in $2^{q-1} - 1$ ways. With the 33 departments, 4,294,967,295.
- **The shortcut:** in regression (and two-class classification), sort the categories by the mean of y and try only the $q - 1$ splits in that order; the best split is among them (Breiman et al., 1984; ESL, section 9.2.4). That is what the tree did with the departments.
- **The risk:** a variable with many categories has many chances to split well by luck, and tends to get picked too often. It returns in week 6, with variable importance.
- **Missing data:** `rpart` uses surrogate splits (another variable that mimics the split); `xgboost` learns, at each split, which side to send the missing values.

A5. Other tree ensembles

- **Extra-trees** (Geurts, Ernst, and Wehenkel, 2006): the threshold of each split is also drawn at random. Trees even less alike, at the cost of more bias.
- **BART** (Chipman, George, and McCulloch, 2010; ISL, section 8.2.4): a sum of K small trees; at each iteration every tree is perturbed (one more split, one fewer, new leaf values) toward its partial residual. It is an MCMC: after a burn-in period you average, and the percentiles give the uncertainty. It works well almost without tuning.
- **Quantile forests** (Meinshausen, 2006): each leaf stores its y's, not just their mean; quantiles are predicted and, with them, intervals.
- **Honest and causal forests** (Wager and Athey, 2018): one half of the sample picks the splits and the other estimates the leaf values; they are used to estimate heterogeneous effects. It is the bridge to session 8's causal inference.

A6. The knobs in R

Package	Knob	What it controls	Today
`rpart`	`cp`	the price per leaf, α divided by the root's RSS	0, then prune
	`minsplit`, `minbucket`	minimum size to split and per leaf	20 and 7
	`xval`	the cross-validation folds	week 3's
`randomForest`	`ntree`	B	300 or 500
	`mtry`	m	$p/3$
	`nodesize`	minimum leaf size	5 (1 in classification)
`xgboost`	`nrounds`, `early_stopping_rounds`	B, and how many rounds without improvement to wait	up to 10,000; 50
	`eta`	η, the rate	0.05
	`max_depth`	d, the depth	6
	`subsample`, `colsample_bytree`	fraction of rows and of columns per tree	0.8 and 0.8
	`lambda`, `gamma`	the Ridge on the leaves and the price per leaf (A3)	1 and 0

A7. The out-of-bag error: a test that comes for free

Each tree in the forest leaves out roughly a third of the stations (ISL, section 8.2.1)

- Each tree's sample is drawn with replacement and leaves out, on average, 36.8 % of the stations: $(1 - 1/n)^n \to e^{-1}$. With 300 trees, each station is left out ("out of the bag", OOB) of about 110 of them.
- **A station's out-of-bag prediction** averages only the trees that never saw it: it is honest, like a cross-validation fold's.
- With $m = 12$: 15.9 out of bag and 15.6 on the test set. With no folds and no refits, the forest brings its own validation; with large B, it resembles LOOCV.

Golden rule #3, again

The out-of-bag error validates the forest, not what was done *before* it: if the variables were chosen looking at all the data, it lies too.

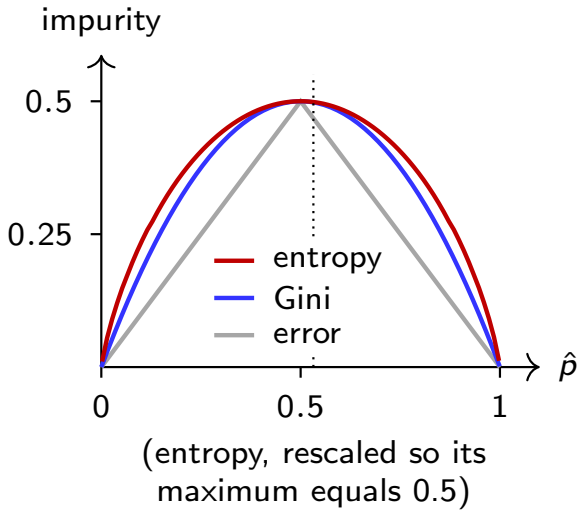
A8. Three impurity measures: error, Gini, and entropy

Classification trees: three ways to measure how mixed a group ends up (ISL, section 8.1.2)

- In each leaf, the fraction of stations Petro won, $\hat{p}$, is week 4's probability; with costs, the decision uses τ^*.
- A split is scored by the **impurity** of the two halves, weighted by their size:

$$\text{error: } E = 1 - \max(\hat{p}, 1 - \hat{p})$$
$$\text{Gini: } G = 2\,\hat{p}\,(1 - \hat{p})$$
$$\text{entropy: } D = -\hat{p}\log\hat{p} - (1 - \hat{p})\log(1 - \hat{p})$$

- At the root, $\hat{p} = 0.53$ and $G = 0.498$. The first split (Afro share at 1.4 %) leaves 0.41 below and 0.75 above: the weighted Gini falls to 0.444; the error, from 0.468 to 0.353.



(entropy, rescaled so its maximum equals 0.5)

Why Gini or entropy for growing: they are curved and reward purer leaves even when the majority class does not change; the error is piecewise flat and misses those splits. For pruning, if what matters is accuracy, the error (ISL).

Appendix references

- Friedman, J. (2002). Stochastic Gradient Boosting. *Computational Statistics & Data Analysis*, 38(4), 367–378. [A2]
- Geurts, P., Ernst, D., & Wehenkel, L. (2006). Extremely Randomized Trees. *Machine Learning*, 63(1), 3–42. [A5]
- Meinshausen, N. (2006). Quantile Regression Forests. *Journal of Machine Learning Research*, 7, 983–999. [A5]
- Chipman, H., George, E., & McCulloch, R. (2010). BART: Bayesian Additive Regression Trees. *Annals of Applied Statistics*, 4(1), 266–298. [A5]
- Wager, S., & Athey, S. (2018). Estimation and Inference of Heterogeneous Treatment Effects using Random Forests. *Journal of the American Statistical Association*, 113(523), 1228–1242. [A5]